International Institute of Information Technology, Bhubaneswar

Department of Computer Science and Engineering

Object Oriented Programming Laboratory

Lab 7 & Lab 8 – Inheritance and Function Overriding (Group 1)

Date: 30.10.2026 **Semester:** B.Tech 3rd Semester **Group:** 1

Instructions

- Write and execute the C++ program for each problem.
- Questions 1–5 cover inheritance; Questions 6–10 cover function overriding.
- Use appropriate classes, constructors, and inheritance structures wherever required.
- Match function signatures in the base and derived classes when overriding.
- Use virtual inheritance and virtual functions wherever specified.
- Display clear and meaningful output.
- Save all programs in your assigned GitHub repository.
- The repository submission must be completed by 12:00 AM on the same day to receive marks.
- Submit the lab record in the prescribed format.

Lab 7 – Inheritance

1. **Railway Ticket Fare – Single Inheritance** Create a base class `Ticket` containing the passenger's name and distance travelled in kilometres. Derive a class `RailwayTicket` containing the fare per kilometre. Calculate the ticket fare. If the distance exceeds 300 km, apply a 10% discount to the calculated fare. Use a constructor to initialize the data members and display the passenger details and final fare.

2. **Digital Image Metadata – Multilevel Inheritance** Create the following hierarchy:

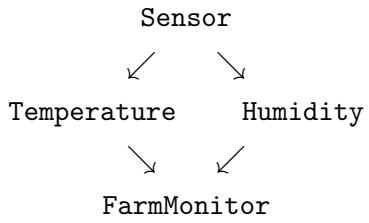
   `Image` → `CompressedImage` → `EncryptedImage`

   The class `Image` stores image width and height in pixels. The class `CompressedImage` stores the compression ratio as a percentage. The class `EncryptedImage` stores the encryption algorithm name. Calculate the original number of pixels and the effective pixel-equivalent size after compression. Display all image metadata. ***Hint:*** *Calculate the effective size as the original pixel count multiplied by the compression percentage divided by 100.*

3. **Examination Eligibility – Hierarchical Inheritance** Create a base class `Candidate` containing a candidate ID and attendance percentage. Derive two classes, `RegularCandidate` and `ReappearCandidate`. A regular candidate is eligible if attendance is at least 75%. A reappear candidate is eligible if attendance is at least 60%. Implement a function in each derived class to check eligibility and display the result.

4. **University Resource Access – Multiple Inheritance** Create two base classes, `LibraryAccess` and `LaboratoryAccess`. The class `LibraryAccess` stores the number of books currently borrowed, while `LaboratoryAccess` stores the number of pending lab submissions. Derive a class `StudentAccess` from both classes. A student receives full resource access only if they have borrowed no more than five books and have no pending lab submissions. Display the student's access status. Use constructors to initialize all data members.

5. **Smart Agriculture – Hybrid Inheritance** Design the following hierarchy:



The class `Sensor` stores a sensor ID. The class `Temperature` stores the temperature in Celsius. The class `Humidity` stores the relative humidity percentage. The class `FarmMonitor` displays all sensor information and issues a warning if the temperature exceeds 35 degrees Celsius or the humidity exceeds 80%. Use virtual inheritance so that `FarmMonitor` contains only one copy of `Sensor`.

Lab 8 – Function Overriding

6. **Document Export System** Create a base class `Document` with a virtual function `exportFile()`. Derive classes `PDFDocument`, `TextDocument`, and `CSVDocument`. Override `exportFile()` in each class to display the appropriate export format and the corresponding file extension. Create objects of all three classes and invoke the overridden function through base-class pointers.

7. **Electricity Bill Estimator** Create a base class `ElectricityPlan` with a virtual function `calculateBill(double units)`. Derive classes `ResidentialPlan` and `CommercialPlan`. For `ResidentialPlan`, charge Rs. 4 per unit for the first 100 units and Rs. 6 per additional unit. For `CommercialPlan`, charge Rs. 7 per unit for all units. Override `calculateBill()` in both derived classes. Use a base-class pointer to calculate and display the bills for both plans.

8. **Autonomous Traffic Signal** Create a base class `TrafficSignal` with a virtual function `getGreenDuration(int vehicleCount)`. Derive classes `UrbanSignal` and `HighwaySignal`. For `UrbanSignal`, calculate the green duration as $10 + 2\times$ vehicle count, capped at 60 seconds. For `HighwaySignal`, calculate the green duration as $15 + 3\times$ vehicle count, capped at 90 seconds. Override the function in both derived classes. Display the calculated green duration for each signal using base-class pointers.

9. **Data Backup Policy** Create a base class `BackupPolicy` with a virtual function `calculateStorage()` that accepts a parameter of type `double` representing the data size in MB. Derive classes `FullBackup` and `IncrementalBackup`. Override the function in both classes according to the following rules:

   - `FullBackup`: Required storage is 100% of the given data size.
   - `IncrementalBackup`: Required storage is 25% of the given data size, plus an additional fixed overhead of 50 MB.

   Use a base-class pointer to invoke the overridden function for both backup policies. Display the required storage for each policy.

10. **Scholarship Selection System** Create a base class `Scholarship` with a virtual function `calculateAward(double marks, double familyIncome)`. Derive classes `MeritScholarship` and `NeedBasedScholarship`. For `MeritScholarship`, award Rs. 20,000 if marks are at least 90; otherwise, award Rs. 10,000 if marks are at least 80; otherwise, award Rs. 0. For `NeedBasedScholarship`, award Rs. 25,000 if family income is below Rs. 2,00,000 and marks are at least 70; otherwise, award Rs. 0. Override the function in both derived classes. Use a base-class pointer to calculate and display the scholarship award under each scheme.